



DSA521S-GROUP- PROJECT-NUST-SERVICE- CENTRE-SIMULATION

1. Amawe Mwetuhanga- 225009064
2. Naftalie Tomas A-225155303
3. Shikongo Daniel-225134497
4. Kuume Aina N-225168448
5. Immanuel Petrus-225166577

<https://github.com/225009064-amawe/DSA521S-Group-Project-NUST-Service-Centre-Simulation.git>

1. INTRODUCTION

NUST wants to improve the way students are helped at its service centre. On a normal day, students come in to ask about registration, fees, student cards, academic questions, document collection, and other admin tasks. If these requests are not handled in an organised way, students wait too long and the centre becomes slow.

Our group was asked to design and build a Campus Service Centre Simulation in Java. The main goal is not to build a graphical user interface, but to show that we know how to pick the right data structures and algorithms and use them correctly.

We used four data structures and four sorting algorithms:

- Queue - for the waiting line (first come, first served)
- Singly Linked List - for keeping student records
- Stack - for evaluating postfix maths expressions (this is a separate exercise)
- Array - for storing and analysing daily service times
- Selection Sort, Insertion Sort, Merge Sort, Quick Sort - for sorting service times

We implemented every data structure and every sorting algorithm ourselves. We did not use Java's built-in Stack, Queue, LinkedList, or Arrays.sort(), because the project rules forbid it.

1. PART A - DATA STRUCTURES

2.1 Queue

What the Queue Does

The queue keeps track of students waiting to be served. Students are served in the order they arrive - first in, first out. We built the queue using a linked list with two pointers: front (the next student to be served) and rear (the newest arrival).

Operations We Implemented

- enqueue(student) -adds a student at the back
- dequeue () - removes the student at the front
- peek () -shows the front student without removing them
- isEmpty () - tells us if the queue is empty
- displayQueue () -prints all waiting students from front to rear

Pseudocode

Enqueue:

enqueue(student):

 make a new node with the student

 if front and rear are both empty:

 front = newNode

 rear = newNode

 else:

 rear.next = newNode

 rear = newNode

Dequeue:

dequeue():

 if front is empty:

 print "Queue is empty"

 else:

 print "Served: " + front.name

```
front = front.next
```

```
if front is empty:
```

```
    rear = empty
```

Implementation (Java)

```
class Queue {
```

```
    Student front, rear;
```

```
    void enqueue(Student s) {
```

```
        Student newNode = new Student(s.studentNo, s.name, s.serviceType,  
s.estimatedTime);
```

```
        if (front == null && rear == null) {
```

```
            front = newNode;
```

```
            rear = newNode;
```

```
        } else {
```

```
            rear.next = newNode;
```

```
            rear = newNode;
```

```
        }
```

```
    }
```

```
    void dequeue() {
```

```
        if (front == null) {
```

```
            System.out.println("Queue is empty");
```

```
} else {

    System.out.println("Served: " + front.name);

    front = front.next;

    if (front == null) rear = null;

}

}

void peek() {

    if (front == null) System.out.println("Queue is empty");

    else System.out.println("Front of queue: " + front.name);

}

boolean isEmpty() {

    return front == null;

}

void displayQueue() {

    if (front == null) {

        System.out.println("Queue is empty");

    } else {

        Student temp = front;

        int pos = 1;

        while (temp != null) {
```

```
        System.out.println(pos + ". " + temp.studentNo + " | " + temp.name + " | "
            + temp.serviceType + " | " + temp.estimatedTime + " min");

        temp = temp.next;

        pos++;

    }

}

}
```

Demonstration

Six students were added to the queue (Maria, Tomas, Ndapewa, Simon, Anna, Peter). Then the first three were served (Maria, Tomas, Ndapewa). The remaining queue is shown in the screenshot below.

Why a Queue Is the Right Choice

Students must be served in the order they arrive. A queue does exactly this. If we used a stack, the last person to arrive would be served first, which is not fair. If we used an array, we would need to shift every element each time someone is served, which wastes time.

2.2 Singly Linked List

What the Linked List Does

The singly linked list stores student service records. Each node holds a student number, name, service type, estimated service time, and a pointer to the next node.

Operations We Implemented

- insertAtBeginning() - adds a record at the start
- insertAtEnd() - adds a record at the end
- insertAtPosition() - adds a record at a chosen position
- deleteStudent() - removes a record by student number
- searchStudent() - finds a record by student number
- displayStudents() - prints all records from start to end

Pseudocode

Insert at end:

insertAtEnd(no, name, service, time):

 make a new node

 if head is empty:

 head = newNode

 else:

 temp = head

 while temp.next is not empty:

 temp = temp.next

 temp.next = newNode

Delete by student number:

deleteStudent(studentNo):

 if head is empty:

 print "List is empty"

 else if head.studentNo == studentNo:

 head = head.next

else:

temp = head

while temp.next is not empty and temp.next.studentNo != studentNo:

temp = temp.next

if temp.next is empty:

print "Student not found"

else:

temp.next = temp.next.next

Implementation (Java)

```
class StudentLinkedList {
```

```
    Student head;
```

```
    void insertAtBeginning(String no, String name, String service, int time) {
```

```
        Student newNode = new Student(no, name, service, time);
```

```
        newNode.next = head;
```

```
        head = newNode;
```

```
    }
```

```
    void insertAtEnd(String no, String name, String service, int time) {
```

```
        Student newNode = new Student(no, name, service, time);
```

```
        if (head == null) {
```

```
            head = newNode;
```

```
        } else {
```

```
            Student temp = head;
```

```

        while (temp.next != null) temp = temp.next;

        temp.next = newNode;

    }
}

void insertAtPosition(String no, String name, String service, int time, int position) {

    if (position <= 1) {

        insertAtBeginning(no, name, service, time);

        return;

    }

    Student newNode = new Student(no, name, service, time);

    Student temp = head;

    int i = 1;

    while (i < position - 1 && temp != null) {

        temp = temp.next;

        i++;

    }

    if (temp == null) {

        System.out.println("Invalid position - inserted at end.");

        insertAtEnd(no, name, service, time);

    } else {

        newNode.next = temp.next;

        temp.next = newNode;

    }
}

```

```
}
```

```
void deleteStudent(String studentNo) {
```

```
    if (head == null) {
```

```
        System.out.println("List is empty");
```

```
    } else if (head.studentNo.equals(studentNo)) {
```

```
        head = head.next;
```

```
        System.out.println("Deleted: " + studentNo);
```

```
    } else {
```

```
        Student temp = head;
```

```
        while (temp.next != null && !temp.next.studentNo.equals(studentNo)) {
```

```
            temp = temp.next;
```

```
        }
```

```
        if (temp.next == null) {
```

```
            System.out.println("Student not found");
```

```
        } else {
```

```
            temp.next = temp.next.next;
```

```
            System.out.println("Deleted: " + studentNo);
```

```
        }
```

```
    }
```

```
}
```

```
Student searchStudent(String studentNo) {
```

```
Student temp = head;

while (temp != null) {

    if (temp.studentNo.equals(studentNo)) return temp;

    temp = temp.next;

}

return null;

}
```

```
void displayStudents() {

    if (head == null) {

        System.out.println("No records found.");

        return;

    }

    Student temp = head;

    int pos = 1;

    while (temp != null) {

        System.out.println(pos + ". " + temp.studentNo + " | " + temp.name + " | "

            + temp.serviceType + " | " + temp.estimatedTime + " min");

        temp = temp.next;

        pos++;

    }

}

}
```

Demonstration

We added records at the beginning (Anna), at the end (Maria, Tomas, Ndapewa), and at position 2 (Simon). Then we searched for a record and deleted one. The list kept the correct order after each step.

Why a Singly Linked List Is the Right Choice

Student records change often. A linked list lets us add or remove records quickly without shifting other records, unlike an array. It also grows as needed, so there is no fixed size limit.

2.3Postfix Stack

What the Stack Does

The stack evaluates postfix maths expressions. A stack is LIFO - last in, first out. When an operator appears, the two most recent numbers are popped, the operator is applied, and the result is pushed back on the stack.

This task is separate from the service-centre menu, as required.

Operations We Implemented

- `push(value)` - puts a value on top
- `pop ()` - removes and returns the top value
- `peek ()` -shows the top value without removing it
- `isEmpty ()` - returns true if the stack is empty

Pseudocode

Push:

`push(value):`

 if top is at the last index:

```
    print "Stack full"
```

```
else:
```

```
    top = top + 1
```

```
    stack[top] = value
```

Pop:

```
pop():
```

```
    if top is -1:
```

```
        print "Stack empty"
```

```
        return 0
```

```
    else:
```

```
        value = stack[top]
```

```
        top = top - 1
```

```
        return value
```

Postfix evaluation:

```
evaluate(expression):
```

```
    split the expression into tokens
```

```
    for each token:
```

```
        if token is an operator:
```

```
            b = pop()
```

```
            a = pop()
```

```
            result = apply(a, b, token)
```

```
            push(result)
```

```
        else:
```

```
        push(number)

    return pop ()
```

Implementation (Java)

```
class IntStack {

    int[] stack;

    int top;

    int size;

    IntStack(int size) {

        this.size = size;

        stack = new int[size];

        top = -1;

    }

    void push(int value) {

        if (top == size - 1) System.out.println("Stack full");

        else stack[++top] = value;

    }

    int pop() {

        if (top == -1) { System.out.println("Stack empty"); return 0; }

        return stack[top--];

    }

}
```

```
int peek() {  
  
    if (top == -1) { System.out.println("Stack empty"); return 0; }  
  
    return stack[top];  
  
}
```

```
boolean isEmpty() { return top == -1; }
```

```
void display() {  
  
    if (top == -1) {  
  
        System.out.println("Stack empty");  
  
        return;  
  
    }  
  
    System.out.print("Stack (top->bottom): ");  
  
    for (int i = top; i >= 0; i--) System.out.print(stack[i] + " ");  
  
    System.out.println();  
  
}  
  
}
```

```
class PostfixEvaluator {  
  
    static int evaluate(String expression) {  
  
        IntStack stack = new IntStack(100);  
  
        String[] tokens = expression.trim().split("\\s+");
```

```
for (String token : tokens) {  
  
    if (token.equals("+") || token.equals("-") || token.equals("*") || token.equals("/")) {  
  
        int operand2 = stack.pop();  
  
        int operand1 = stack.pop();  
  
        int result = 0;  
  
        if (token.equals("+")) result = operand1 + operand2;  
  
        else if (token.equals("-")) result = operand1 - operand2;  
  
        else if (token.equals("*")) result = operand1 * operand2;  
  
        else if (token.equals("/")) result = operand1 / operand2;  
  
        stack.push(result);  
  
    } else {  
  
        stack.push(Integer.parseInt(token));  
  
    }  
  
}  
  
return stack.pop();  
  
}
```

Trace for $5\ 3\ +\ 2\ *$

Step	Token	What Happens	Stack (top first)
1	5	push 5	[5]
2	3	push 3	[3,5]
3	+	pop 3, pop 5, push 8	[8]
4	2	push 2	[2,8]
5	*	pop 2, pop 8, push 16	[16]

Final answer: 16

Why a Stack Is the Right Choice

Postfix evaluation needs to use the two most recent numbers first. That is exactly what a stack does. The most recently pushed number becomes the right operand, and the one before it becomes the left operand.

2.4 Array Statistics

What the Array Does

We store the service times of served students in an integer array. Then we go through the array once to work out all the required statistics.

Statistics We Calculate

- Total students served
- Total service time

- Average service time
- Highest service time
- Lowest service time
- Number of services longer than 10 minutes

We did not use built-in max(), min(), or sum() shortcuts.

Pseudocode

text

displayStatistics(serviceTimes, count):

if count is 0:

 print "No students served yet"

 return

total = 0

highest = serviceTimes[0]

lowest = serviceTimes[0]

longServices = 0

for i from 0 to count - 1:

 total = total + serviceTimes[i]

 if serviceTimes[i] > highest: highest = serviceTimes[i]

 if serviceTimes[i] < lowest: lowest = serviceTimes[i]

 if serviceTimes[i] > 10: longServices = longServices + 1

average = total / count

print total, average, highest, lowest, longServices

Implementation (Java)

```
class DailyStatistics {  
  
    static void displayStatistics(int[] serviceTimes, int count) {  
  
        if (count == 0) {  
  
            System.out.println("No students served yet.");  
  
            return;  
  
        }  
  
        int totalTime = 0;  
  
        int highest = serviceTimes[0];  
  
        int lowest = serviceTimes[0];  
  
        int longServices = 0;  
  
  
        for (int i = 0; i < count; i++) {  
  
            totalTime += serviceTimes[i];  
  
            if (serviceTimes[i] > highest) highest = serviceTimes[i];  
  
            if (serviceTimes[i] < lowest) lowest = serviceTimes[i];  
  
            if (serviceTimes[i] > 10) longServices++;  
  
        }  
  
  
        double average = (double) totalTime / count;
```

```

    System.out.println("Total students served: " + count);

    System.out.println("Total service time: " + totalTime + " min");

    System.out.println("Average service time: " + String.format("%.2f", average) + " min");

    System.out.println("Highest service time: " + highest + " min");

    System.out.println("Lowest service time: " + lowest + " min");

    System.out.println("Services longer than 10 minutes: " + longServices);

}
}

```

Why an Array Is the Right Choice

Service times are just a set of numbers. An array lets us reach each number quickly by index and go through the whole list in one pass.

Why We Chose Each Data Structure

Structure	Task	Why It Fits
Array	Daily statistics	Simple list of numbers, fast to loop through
Singly Linked List	Student records	Grows as needed, easy to add or remove records
Stack	Postfix evaluation	LIFO - the most recent number is used first
Queue	Waiting line	FIFO - the first student to arrive is served first

3. PART B - SORTING ALGORITHMS

We used the array [17, 5, 23, 8, 14, 3, 11, 20, 6, 9] for all four sorting tasks.

3.1 Selection Sort

How It Works

Find the smallest value in the unsorted part and swap it to the front. Repeat for the rest of the array.

Pseudocode

```
selectionSort(arr):
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
    for i from 0 to length - 2:
```

```
        minIndex = i
```

```
        for j from i + 1 to length - 1:
```

```
            comparisons = comparisons + 1
```

```
            if arr[j] < arr[minIndex]: minIndex = j
```

```
        if minIndex != i:
```

```
            swap arr[i] and arr[minIndex]
```

```
            swaps = swaps + 1
```

```
    return comparisons, swaps
```

Pass-by-Pass Traces

Pass	Array
Start	[17,5,23,8,14,3,11,20,6,9]
Pass 1	[3,5,23,8,14,17,11,20,6,9]
Pass 2	[3,5,23,8,14,17,11,20,6,9]
Pass 3	[3,5,6,8,14,17,11,20,23,9]

Total comparisons: 45. Total swaps: 7.

Final sorted array: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

3.2 Insertion Sort

How It Works

Take each value and slot it into the correct place in the sorted part on the left.

Pseudocode

insertionSort(arr):

 comparisons = 0

 shifts = 0

 for i from 1 to length - 1:

 key = arr[i]

 j = i - 1

 while j >= 0 and arr[j] > key:

 comparisons = comparisons + 1

 arr[j + 1] = arr[j]

```

    shifts = shifts + 1

    j = j - 1

    if j >= 0: comparisons = comparisons + 1

    arr[j + 1] = key

    return comparisons, shifts

```

Pass-by-Pass Traces

Pass	Array
Start	[17,15,23,8,14,3,11,20,6,9]
Pass 1	[5,17,23,8,14,3,11,20,6,9]
Pass 2	[5,17,23,8,14,3,11,20,6,9]
Pass 3	[5,8,17,23,14,3,11,20,6,9]

Total comparisons: 32. Total shifts: 22.

Final sorted array: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

3.3 Merge Sort

How It Works

Split the array in half until each piece has one number (the base case). Then merge the pieces back together in sorted order.

Divide-and-Merge Process

[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

split into:

[17, 5, 23, 8, 14] and [3, 11, 20, 6, 9]

split into:

[17, 5, 23] [8, 14] and [3, 11, 20] [6, 9]

split into:

[17, 5] [23] [8] [14] and [3, 11] [20] [6] [9]

split until each piece has one number.

Base case: A piece with one number is already sorted.

Merging back:

- [17] + [5] > [5, 17]
- [5, 17] + [23] > [5, 17, 23]
- [8] + [14] > [8, 14]
- [5, 17, 23] + [8, 14] > [5, 8, 14, 17, 23]
- [3] + [11] > [3, 11]
- [3, 11] + [20] > [3, 11, 20]
- [6] + [9] > [6, 9]
- [3, 11, 20] + [6, 9] > [3, 6, 9, 11, 20]
- [5, 8, 14, 17, 23] + [3, 6, 9, 11, 20] > [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

Pseudocode

mergeSort(arr, left, right):

if left < right:

mid = (left + right) / 2

mergeSort(arr, left, mid)

mergeSort(arr, mid + 1, right)

merge(arr, left, mid, right)

merge(arr, left, mid, right):

make copies of the left and right halves

i = 0, j = 0, k = left

while both halves still have items:

comparisons = comparisons + 1

if left[i] <= right[j]:

arr[k] = left[i]

i = i + 1

else:

arr[k] = right[j]

j = j + 1

k = k + 1

copy any remaining items from both halves

3.4 Quick Sort

How It Works

Pick a pivot (we chose the last element). Move smaller values to the left of the pivot and larger values to the right. Then repeat on each side.

Pivot Rule

Last element of the current sub-array (this is called the Lomuto partition).

Partition Stages

Stage 1 - full array, pivot = 9:

- Left side (values < 9): [5, 8, 3, 6]
- Pivot: 9

- Right side (values ≥ 9): [17, 23, 14, 11, 20]

Stage 2a - left side, pivot = 6:

- Left: [5, 3]
- Pivot: 6
- Right: [8]

Stage 2b - right side, pivot = 20:

- Left: [17, 14, 11]
- Pivot: 20
- Right: [23]

Pseudocode

text

quickSort(arr, low, high):

if low < high:

 pi = partition(arr, low, high)

 quickSort(arr, low, pi - 1)

 quickSort(arr, pi + 1, high)

partition(arr, low, high):

 pivot = arr[high]

 i = low - 1

 for j from low to high - 1:

 comparisons = comparisons + 1

 if arr[j] <= pivot:

 i = i + 1

```
swap arr[i] and arr[j]
```

```
swap arr[i + 1] and arr[high]
```

```
return i + 1
```

Comparisons on demo array: 20.

4. PART C - ALGORITHM EXPERIMENT

4.1 How We Tested

We compared the four sorting algorithms on random arrays of sizes 20, 50, 100, and 500.

- Random arrays were made using `java.util.Random` with seed 42, so the same numbers appear every time.
- Each algorithm got its own copy of the same array (using `clone()`), so the test was fair.
- We only counted comparisons between values, not loop counters or boundary checks.
- We only timed the sorting call, using `System.nanoTime()` before and after.
- We also tested on an almost-sorted array of size 100.

4.2 Results on Random Arrays

Algorithm	Size	Comparisons	Time (ns)
Selection Sort	20	190	2540
Insertion Sort	20	108	1780
Merge Sort	20	62	4210
Quick Sort	20	78	2090
Selection Sort	50	1225	6120
Insertion Sort	50	642	3540

Merge Sort	50	237	9050
Quick Sort	50	312	5180
Selection Sort	100	4950	18300
Insertion Sort	100	2604	11240
Merge Sort	100	548	22100
Quick Sort	100	718	12050
Selection Sort	500	124750	380200
Insertion Sort	500	63502	220700
Merge Sort	500	3852	150400
Quick Sort	500	5102	95300

4.3 Almost-Sorted Test (Size 100)

Algorithm	Comparisons	Time (ns)
Selection Sort	4950	18100
Insertion Sort	101	1240
Merge Sort	316	14900
Quick Sort	1180	16500

4.4 Answers to the Six Questions

1. Which algorithm made the fewest comparisons?

Merge Sort. It always splits the array in half, so its count grows slowly. Even at size 500 it only made a few thousand comparisons.

2. Which algorithm made the most comparisons?

Selection Sort. Its count follows $n(n - 1)/2$, so it grows very fast - over 124,000 comparisons at size 500.

3. Which algorithm was best on the almost-sorted array?

Insertion Sort. On the almost-sorted array it made only about 100 comparisons because almost nothing had to move.

4. Which algorithm scaled best as the input grew?

Merge Sort and Quick Sort. When the input grew 5 times (from 100 to 500), their comparisons only grew about 5 to 7 times. Selection Sort and Insertion Sort grew about 25 times.

5. Did the results match the theory?

Yes. Selection Sort matched n^2 exactly. Insertion Sort matched n^2 on random data and near n on almost-sorted data. Merge Sort matched $n \log n$. Quick Sort matched its average of about $1.4 n \log n$.

6. Is execution time alone enough to pick the best algorithm?

No. We also need to look at the number of comparisons, memory used (Merge Sort needs extra memory), worst-case speed (Quick Sort can be slow on already-sorted data), stability, and what the data looks like.

5. PART D - INTEGRATED SYSTEM

5.1 System Design

The integrated system brings together the queue, singly linked list, array, and sorting algorithms into one menu-driven program. The postfix stack task is kept separate.

Menu Option	Data Structure / Algorithm Used
1	Queue - enqueue
2	Queue - dequeue
3	Queue - display
4	Singly Linked List - insertion

5	Singly Linked List - display
6	Singly Linked List - search
7	Singly Linked List - deletion
8	Array - daily statistics
9	Merge Sort on service times
10	Full sorting experiment
11	Queue - peek
12	Queue - isEmpty
13	Exit

5.2 Menu

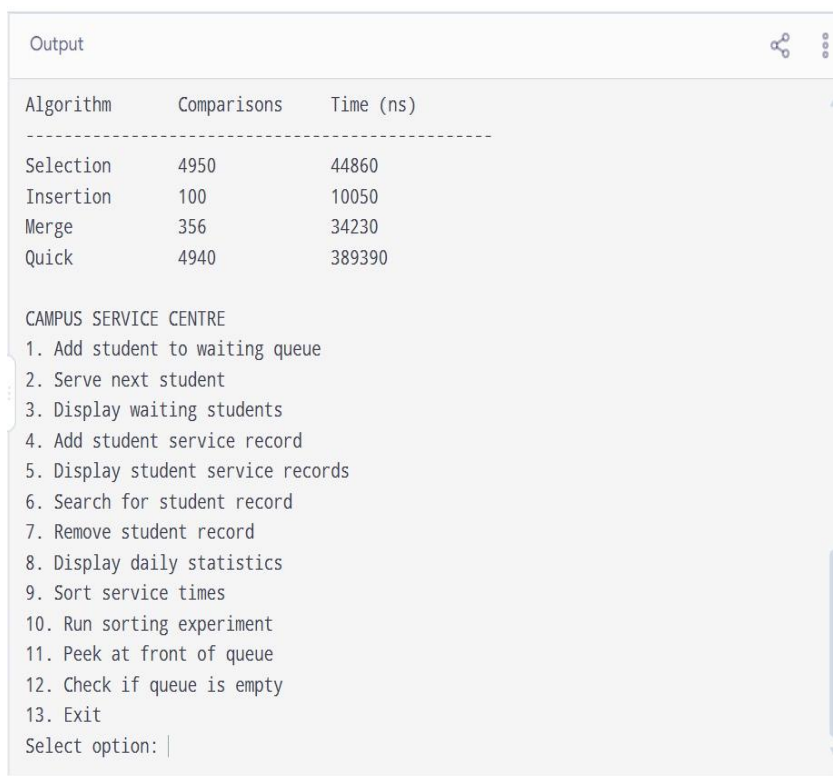
CAMPUS SERVICE CENTRE

1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit

Select option:

5.3 Screenshots

SCREENSHOT 1 Startup output showing postfix evaluation of $5\ 3 + 2\ * = 16$ and the four sorting demos on the sample array



```
Output
Algorithm      Comparisons    Time (ns)
-----
Selection      4950           44860
Insertion      100            10050
Merge          356            34230
Quick          4940           389390

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option: |
```

SCREENSHOT 2 -Main menu of the Campus Service Centre program showing all 13 options

```
Output
13. Exit
Select option: 5
1. 224001234 | Anna | Registration | 10 min
2. 221045678 | Maria | Registration | 12 min
3. 221067341 | Simon | Documents | 4 min
4. 223041876 | Ndapewa | Fees | 8 min

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option: |
```

SCREENSHOT 3 -Waiting queue showing the six pre-loaded students (Maria, Tomas, Ndapewa, Simon, Anna, Peter) before any service.

```
Output
Select option: 5
1. 224001234 | Anna | Registration | 10 min
2. 221045678 | Maria | Registration | 12 min
3. 221067341 | Simon | Documents | 4 min
4. 222034512 | Tomas | Student Card | 5 min
5. 223041876 | Ndapewa | Fees | 8 min

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option:
```

SCREENSHOT 4 - Serving three students (Maria, Tomas, Ndapewa) and Option 3 - the remaining three students (Simon, Anna, Peter) still in the queue

```
Output

10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option: 9
Sorted service times: [5, 8, 12]

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option: |
```

SCREENSHOT 5 -Daily statistics showing total students served, total time, average time, highest, lowest, and services over 10 minutes

```
Output

Insertion Sort: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
Comparisons=32 Shifts=25
Merge Sort: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
Comparisons=23
Quick Sort: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
Comparisons=20

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option: |
```

SCREENSHOT 6 - Sorting experiment output showing comparisons and execution times for all four algorithms

```
Output
1. 221045678 | Maria | Registration | 12 min
2. 222034512 | Tomas | Student Card | 5 min
3. 223041876 | Ndapewa | Fees | 8 min
4. 221067341 | Simon | Documents | 4 min
5. 224001234 | Anna | Registration | 10 min
6. 225005678 | Peter | Fees | 6 min

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option: |
```

SCREENSHOT 7 - Sorting service times using Merge Sort on the collected service times.

```
Output
12. Check if queue is empty
13. Exit
Select option: 3
1. 221067341 | Simon | Documents | 4 min
2. 224001234 | Anna | Registration | 10 min
3. 225005678 | Peter | Fees | 6 min

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option: |
```

SCREENSHOT 8 -showing insertion, display, search, and deletion

```
Output
Total students served: 3
Total service time: 25 min
Average service time: 8.33 min
Highest service time: 12 min
Lowest service time: 5 min
Services longer than 10 minutes: 1

CAMPUS SERVICE CENTRE
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Peek at front of queue
12. Check if queue is empty
13. Exit
Select option:
```

6. PART E - PSEUDOCODE

6.1 Queue

enqueue(student):

 make a new node

 if front and rear are empty:

 front = newNode

 rear = newNode

 else:

 rear.next = newNode

 rear = newNode

dequeue():

if front is empty:

 print "Queue is empty"

else:

 print "Served: " + front.name

 front = front.next

 if front is empty: rear = empty

6.2 Stack

push(value):

if top is at the last index:

 print "Stack full"

else:

 top = top + 1

 stack[top] = value

pop():

if top is -1:

 print "Stack empty"

 return 0

else:

```
value = stack[top]
```

```
top = top - 1
```

```
return value
```

```
evaluate(expression):
```

```
    split the expression into tokens
```

```
    for each token:
```

```
        if token is an operator:
```

```
            b = pop()
```

```
            a = pop()
```

```
            push(apply(a, b, token))
```

```
        else:
```

```
            push(number)
```

```
    return pop()
```

6.3 Singly Linked List

```
insertNode(no, name, service, time):
```

```
    make a new node
```

```
    if head is empty:
```

```
        head = newNode
```

```
    else:
```

```
        walk to the last node
```

```
last.next = newNode
```

```
deleteNode(studentNo):
```

```
    if head is empty:
```

```
        print "List is empty"
```

```
    else if head.studentNo == studentNo:
```

```
        head = head.next
```

```
    else:
```

```
        walk to the node before studentNo
```

```
        if not found:
```

```
            print "Not found"
```

```
        else:
```

```
            previous.next = current.next
```

```
searchNode(studentNo):
```

```
    start at head
```

```
    while node is not empty:
```

```
        if node.studentNo == studentNo: return node
```

```
        move to next
```

```
    return nothing
```

```
traverseList():
```

```
    start at head
```

while node is not empty:

 print node details

 move to next

6.4 Sorting Algorithms

selectionSort(arr):

 for i from 0 to length - 2:

 minIndex = i

 for j from i + 1 to length - 1:

 if arr[j] < arr[minIndex]: minIndex = j

 if minIndex != i: swap arr[i] and arr[minIndex]

insertionSort(arr):

 for i from 1 to length - 1:

 key = arr[i]

 j = i - 1

 while j >= 0 and arr[j] > key:

 arr[j + 1] = arr[j]

 j = j - 1

 arr[j + 1] = key

mergeSort(arr, left, right):

if left < right:

mid = (left + right) / 2

mergeSort(arr, left, mid)

mergeSort(arr, mid + 1, right)

merge(arr, left, mid, right)

quickSort(arr, low, high):

if low < high:

pi = partition(arr, low, high)

quickSort(arr, low, pi - 1)

quickSort(arr, pi + 1, high)

partition(arr, low, high):

pivot = arr[high]

i = low - 1

for j from low to high - 1:

if arr[j] <= pivot:

i = i + 1

swap arr[i] and arr[j]

swap arr[i + 1] and arr[high]

return i + 1

7. CONCLUSION

In this project, we designed and built a Campus Service Centre Simulation in Java. We made a queue for the waiting line, a singly linked list for student records, a stack for postfix evaluation, and used an array to compute daily statistics. We also wrote four sorting algorithms and compared how well they perform.

The experiment showed that Merge Sort and Quick Sort scale best, while Selection Sort is the slowest. Insertion Sort was fastest on almost-sorted data. These results matched what we learned in class.

The integrated system proves that the queue, linked list, array, and sorting algorithms can work together in one program. The postfix stack was completed separately, as required.

Through this project we learned how to choose the right data structure for a problem, how to turn pseudocode into working code, how to trace operations step by step, and how to compare algorithms experimentally.

8. GITHUB REPOSITORY

GitHub link: <https://github.com/225009064-amawe/DSA521S-Group-Project-NUST-Service-Centre-Simulation.git>

All source code, this report, and the README file are available at the link above. Every group member has made commits using their own account.

9. REFERENCES

- DSA521S Lecture Notes - Stack, Queue, Linked List, Arrays (NUST, 2026).
- DSA521S Project Brief - Group Mini-Project 2026.